Variables conflicts in JavaScript

Suppose we have an index.html HTML page where you create a variable str in the script tag and display it on the screen:

index.html

```
<html>
  <head>
    <script>
      let str = 'script text';
      alert(str); // shows 'script text'
    </script>
  </head>
  <body>
    ...
  </body>
</html>
```

Suppose we also have a script.js file, which also sets the variable str:

script.js

```
let str = 'file text';
```

Now let our script.js file be linked to the index.html page as follows:

```
<html>
  <head>
    <script>
      let str = 'script text';
    </script>
    <script src="script.js"></script>
    <script>
      alert(str);
    </script>
  </head>
  <body>
    ...
  </body>
</html>
```

Since the variable str exists both in the index.html file and in the script.js file, there will be a conflict in which that variable will win, which is written below, that is, a variable from the script.js file. So our code will output 'file text', not 'script text' as we expect.

The problem is actually very serious. In a real site, you will most often have several files with your scripts, in addition, you will use some third-party plugins. In this case, the variables and functions of one file may conflict with the variables and functions of another file.

Modules using closures in JavaScript

The problem described above is typical for any programming language. The so-called *modules* are used as a solution.

A module is a kind of construction, made in such a way that the variables and functions of this construction are visible only inside it and don't interfere with anyone outside.

There are several types of modules in JavaScript. The simplest - *modules via closures* are created by in-place function call, like this:

```
;(function() {
  // here is a module code
})();
```

Variables and functions created in such a module will not be visible outside of that module:

```
;(function() {
  let str = 'a module variable';

  function func() {
    alert('a module function');
  }
})();

// Here the variables and functions of the
module are not available:
alert(str);
alert(func);
```

Practical usage

Let we have two divs with numbers:

```
<div id="div1">10</div>
<div id="div2">10</div>
```

Let's make it so that on click on the first div, its value is squared, and on click on the second div, its value is cubed

Let's organize our code in the form of two modules:

```
;(function() {
  let elem = document.querySelector('#div1'); // the first div

  function func(num) {
    return num * num; // let's square
  }

  elem.addEventListener('click', function() {
    this.textContent = func(elem.textContent);
  });
})();

;(function() {
  let elem = document.querySelector('#div2'); // the second div

  function func(num) {
    return num * num * num; // let's cube
  }
```

```
19
20    elem.addEventListener('click', function() {
21      this.textContent = func(elem.textContent);
22    });
23  })();
```

Now in each of the modules we can use any variables and functions without fear that they will conflict with other variables and functions of our script.

For example, we store both elements in the variable elem - each in its own variable of its module. If modules were not here, we would have to introduce different variables to store our elements. And with modules, we can safely use our variable without fear that someone will also want to use this variable.

Passing parameters to a module via closures in JavaScript

It is considered good practice not to hardcode any values into the module, but to pass them as a parameter of the module itself (that is, as a parameter of the function called in-place):

```
1  ;(function(arg1, arg2) { // the parameters get into variables
2
3  })(1, 2); // passing some parameters
```

Let's look at an example. Let's have a div with a number and a button:

```
1  <div id="div">3</div>
2  <button id="btn">click me</button>
```

Suppose we also have some module:

```
1  ;(function() {
2    let div = document.querySelector('#div');
3    let btn = document.querySelector('#btn');
4
5    function func(num) {
6      return num * num;
7    }
8
9    btn.addEventListener('click', function() {
10     div.textContent = func(div.textContent);
11   });
12 })();
```

As you can see, our element selectors are hardcoded in the module code. A better solution would be to pass them as module parameters - so in the future we can easily change them. Let's refactor our module:

```
1  ;(function(selector1, selector2) {
2    let div = document.querySelector(selector1);
3    let btn = document.querySelector(selector2);
4
5    function func(num) {
6      return num * num;
7    }
8
9    btn.addEventListener('click', function() {
10     div.textContent = func(div.textContent);
11   });
12 })('#div', '#btn');
```

⊗jsPmMCPP

№ 1

Given a button and three inputs into which numbers are entered. On pressing the button print the sum of the entered numbers to the console. Implement a task using a module.

Passing a parent element to module via closures in JavaScript

Let's say we have the following elements:

```
<div id="div1">1</div>
<div id="div2">2</div>
<div id="div3">3</div>
<div id="res"></div>

<button id="btn">click me</button>
```

Let the following module work with these elements:

```
;(function(selector1, selector2, selector3, selector4, selector5) {
  let div1 = document.querySelector(selector1);
  let div2 = document.querySelector(selector2);
  let div3 = document.querySelector(selector3);
  let res  = document.querySelector(selector4);
  let btn  = document.querySelector(selector5);

  btn.addEventListener('click', function() {
    let num1 = Number(div1.textContent);
    let num2 = Number(div2.textContent);
    let num3 = Number(div3.textContent);

    res.textContent = num1 + num2 + num3;
  });
})('#div1', '#div2', '#div3', '#res', '#btn');
```

As you can see, the number of parameters passed to the module is somewhat large and creates inconvenience. Usually, in this case, they proceed as follows: they pass to the module the common parent of all elements with which our module works, and already inside the module, various elements are obtained from this parent.

Let's make our tags have a common parent:

```
<div id="parent">
  <div id="div1">1</div>
  <div id="div2">2</div>
  <div id="div3">3</div>
  <div id="res"></div>

  <button id="btn">click me</button>
</div>
```

Let's change the module code now:

```
;(function(root) {
  let parent = document.querySelector(root);

  let div1 = parent.querySelector('#div1');
  let div2 = parent.querySelector('#div2');
  let div3 = parent.querySelector('#div3');
```

```
  let res  = parent.querySelector('#res');
  let btn  = parent.querySelector('#btn');

  btn.addEventListener('click', function() {
    let num1 = Number(div1.textContent);
    let num2 = Number(div2.textContent);
    let num3 = Number(div3.textContent);

    res.textContent = num1 + num2 + num3;
  });
})('#parent');
```

Thus, it turns out that we pass the parent element to the module from the outside and can easily change it. And child elements are an internal matter of the module.

Passing module settings via closures in JavaScript

Let's say we have the following module:

```
;(function(root, type, amount) {
  let parent = document.querySelector(root);

  for (let i = 1; i <= amount; i++) {
    let elem = document.createElement(type);
    parent.append(elem);
  }
})('#parent', 'p', 5);
```

As you can see, three settings are passed into this module: the parent element selector, the element type to create, and the number of elements.

As a rule, such settings are made in the form of an object:

```
let config = {
  root:  '#parent',
  type:  'p',
  amount: 5
}
```

Let's pass our object as a module parameter:

```
;(function(config) {
  let parent = document.querySelector(config.root);

  for (let i = 1; i <= config.amount; i++) {
    let elem = document.createElement(config.type);
    parent.append(elem);
  }
})(config);
```

It is more common to destructure an object with settings:

```
;(function({root, type, amount}) {
  let parent = document.querySelector(root);

  for (let i = 1; i <= amount; i++) {
    let elem = document.createElement(type);
    parent.append(elem);
  }
})(config);
```

Default parameters

Suppose we want to allow not specify all settings when using the module. If any of the settings is not specified, then it will take the default value.

For example, in our case, we can make the default type take the value p, and the amount - the value 5:

```
;(function({root, type = 'p', amount = 5}) {
  let parent = document.querySelector(root);

  for (let i = 1; i <= amount; i++) {
    let elem = document.createElement(type);
    parent.append(elem);
  }
})(config);
```

In this case, we can easily configure our module in different ways. For example, let's specify only the parent element:

```
let config = {
  root:  '#parent',
}
```

Now let's specify the parent element and the amount. In this case, we will not need to specify the type - after all, the elements of the settings object have no order, and we can omit them as we like. So here is our setup:

```
let config = {
  root:  '#parent',
  amount: 10
}
```

Exporting variables and functions in modules via closures in JavaScript

Sometimes you need to make certain variables and functions of a module accessible from the outside. Let's see how it's done. Let's say we have the following module:

```
;(function() {
  let str = 'a module variable';

  function func() {
    alert('a module function');
  }
})();
```

Let's export our function func. To do this, we write it into the property of the window object embedded in the browser:

```
;(function() {
  let str = 'a module variable';

  function func() {
    alert('a module function');
  }

  window.func = func;
})();
```

Now we can call our function from outside the module:

```
;(function() {
  let str = 'a module variable';

  function func() {
    alert('a module function');
  }

  window.func = func;
})();

window.func(); // shows 'a module function'
```

In this case, it is not necessary to call the function as a property of the window object:

```
;(function() {
  let str = 'a module variable';

  function func() {
    alert('a module function');
  }

  window.func = func;
})();

func(); // shows 'a module function'
```

⊗jsPmMCVFE

№1

Given the following module:

```
;(function() {
  let str1 = 'a module variable';
  let str2 = 'a module variable';
  let str3 = 'a module variable';

  function func1() {
    alert('a module function');
  }
  function func2() {
    alert('a module function');
  }
  function func3() {
    alert('a module function');
  }
})();
```

Export one of the variables and any two functions to the outside.

Exporting an Object in Modules via Closures in JavaScript

Let's say we have the following module:

```
;(function() {
  function func1() {
    alert('module funcion');
  }
  function func2() {
    alert('module funcion');
  }
  function func3() {
    alert('module funcion');
  }
})();
```

Let's say we want to export all three functions externally. In this case, to avoid cluttering the outside of the module with extra function names, it's better to place all the functions into a single object and export that object:

```
;(function() {
  function func1() {
    alert('module funcion');
  }
  function func2() {
    alert('module funcion');
  }
  function func3() {
    alert('module funcion');
  }

  window.module = {func1: func1, func2: func2, func3: func3};
})();
```

Since the key names and variable names match, the object with functions can be simplified:

```
;(function() {
  function func1() {
    alert('module funcion');
  }
```

```
  function func2() {
    alert('module funcion');
  }
  function func3() {
    alert('module funcion');
  }

  window.module = {func1, func2, func3};
})();
```

Another approach is also possible. We can assign the functions to the object immediately when declaring them, like this:

```
;(function() {
  let module = {};

  module.func1 = function() {
    alert('module funcion');
  }
  module.func2 = function() {
    alert('module funcion');
  }
  module.func3 = function() {
    alert('module funcion');
  }

  window.module = module;
})();
```

№1

⊗jsPmMCVFEO

Given the following module:

```
;(function() {
  let str1 = 'module variable';
  let str2 = 'module variable';
  let str3 = 'module variable';

  function func1() {
    alert('module funcion');
  }
  function func2() {
    alert('module funcion');
  }
  function func3() {
    alert('module funcion');
  }
  function func4() {
    alert('module funcion');
  }
  function func5() {
    alert('module funcion');
  }
})();
```

Export an object containing the first five functions and the first two variables externally.

Libraries via closures in JavaScript

Often they create JavaScript *libraries*, which are collections of functions for other programmers to use.

Such libraries are usually wrapped into modules using closures. This is done so that when the library is linked, as few functions as possible appear in the outside evironment.

As a rule, each library tries to create only one variable in the outside environment - an object with library functions. At the same time, inside the library code, some of the functions are basic, and some are helper. Obviously, we want to export only the necessary functions to the outside environment, without littering the exported object with helper functions.

Let's look at an example. Suppose we have the following set of functions that we would like to turn into a library:

math.js

```
function square(num) {
  return num ** 2;
}
function cube(num) {
  return num ** 3;
}
function avg(arr) {
  return sum(arr, 1) / arr.length;
}
function digitsSum(num) {
  return sum(String(num).split(''));
}

// helper function
function sum(arr) {
  let res = 0;

  for (let elem of arr) {
    res += +elem;
  }

  return res;
}
```

Let's make our functions look like a module:

math.js

```
;(function() {
  function square(num) {
    return num ** 2;
  }
  function cube(num) {
    return num ** 3;
  }
  function avg(arr) {
    return sum(arr, 1) / arr.length;
  }
  function digitsSum(num) {
    return sum(String(num).split(''));
  }

  // helper function
  function sum(arr) {
    let res = 0;
```

```
18
19      for (let elem of arr) {
20        res += +elem;
21      }
22
23      return res;
24    }
25  })();
```

And now we export all functions, except for the helper one:

math.js

```
1   ;(function() {
2     function square(num) {
3       return num ** 2;
4     }
5     function cube(num) {
6       return num ** 3;
7     }
8     function avg(arr) {
9       return sum(arr, 1) / arr.length;
10    }
11    function digitsSum(num) {
12      return sum(String(num).split(''));
13    }
14
15    // helper function
16    function sum(arr) {
17      let res = 0;
18
19      for (let elem of arr) {
20        res += +elem;
21      }
22
23      return res;
24    }
25
26    window.math = {square, cube, avg, digitsSum};
27  })();
```

Let's say we have an index.html HTML page :

index.html

```
1   <html>
2     <head>
3       <script>
4
5       </script>
6     </head>
7   </html>
```

Let's link our library to it:

index.html

```
1   <html>
2     <head>
3       <script src="math.js"></script>
4       <script>
5
```

```
    </script>
  </head>
</html>
```

Let's use functions from our library:

index.html

```
<html>
  <head>
    <script src="math.js"></script>
    <script>
      alert(math.avg([1, 2, 3]) + math.square());
    </script>
  </head>
</html>
```

№1

⊗jsPmMCLb

Given the following code:

```
function avg1(arr) {
  return sum(arr, 1) / arr.length;
}

function avg2(arr) {
  return sum(arr, 2) / arr.length;
}

function avg3(arr) {
  return sum(arr, 3) / arr.length;
}

// helper function
function sum(arr, pow) {
  let res = 0;

  for (let elem of arr) {
    res += elem ** pow;
  }

  return res;
}
```

Make this code in the form of a module. Export all functions except the helper one to the outside.

№2

⊗jsPmMCLb

Study the underscore library. Make your own similar library by repeating 5-10 functions of the original library in it.